

FAROS Idea Module Plan v5: 六步双图谱 + BFTS 搜索树设计

一、版本目标

v5 的目标是把 idea 模块重新压回图二的六步骤，同时吸收图一的思路：

```
expandQuery
-> literatureSearch
-> noveltyCheck
-> gapAnalysis
-> ideaBrainstorm
-> rankCandidates
```

新的职责划分：

Step 1: expandQuery	任务理解和搜索种子生成
Step 2: literatureSearch	构建 Graph 1 的宽文献池和论文级图
Step 3: noveltyCheck	在 Graph 1 上做论文筛选、深读和 novelty evidence
Step 4: gapAnalysis	构建 Graph 2，并把 Graph 1 的 gap/frontier 映射进去
Step 5: ideaBrainstorm	接入 AI Scientist v2 风格 BFTS 搜索树
Step 6: rankCandidates	多维排序、critique、交付最终候选

一句话版本：

第二到第四步主要完成双图谱构建，第五步从 Graph 2 接入 BFTS idea search tree，把迭代、反思、文献检索循环都放进搜索树里完成。

v5 不再把 reasoningPathDiscovery 单独拆成第七步。Graph 2 的路径种子由 Step 4 产出，Step 5 把这些路径种子作为搜索树 root / initial nodes 使用。

二、核心架构

2.1 双图谱定位

Graph 1: Literature Graph
粒度: paper-level
作用: 文献覆盖、聚类、筛选、frontier/gap evidence
生命周期: Step 2 创建, Step 3 精炼

Graph 2: Reasoning KG
粒度: concept-level
作用: 抽取方法、问题、机制、数据集、指标、发现, 并形成可推理路径
生命周期: Step 4 创建, Step 5 搜索树可增量补丁

两个图谱不是并列生成后简单拼接, 而是有明确交接关系:

Graph 1 先回答: 哪些论文值得作为证据基础?
Graph 2 再回答: 这些证据能组合出哪些 hypothesis path?
BFTS 最后回答: 哪些 path 值得经过反思、检索和变异成长为候选 idea?

2.2 六步总流程



2.3 与 v4 的关键变化

维度	v4	v5
主流程	7 步, 单独有 Reasoning Path Discovery	回到图二 6 步

维度	v4	v5
Step 2-4	文献图、深读、KG 分散表达	明确定义为双图谱构建阶段
Step 5	生成候选 idea	BFTS 搜索树，负责迭代、反思、文献检索、变异
Graph 2 路径	Step 5 发现路径	Step 4 产出 path seeds, Step 5 扩展
文献检索循环	主要在前置文献阶段	前置检索 + 搜索树内 LiteratureProbe 增量检索
idea 生成	path 翻译为 candidate	search node 逐轮成长为 candidate

三、Step 1: expandQuery

3.1 目标

Step 1 仍然保留 `expandQuery` 名称，但它不只是扩展关键词，而是生成后续建图和搜索树的任务计划。

它回答：

1. 用户真正想探索的研究对象是什么？
2. `paperType` 应该采用哪种路径模板？
3. Graph 1 检索应该覆盖哪些 query family？
4. Graph 2 应该优先关注哪些实体类型？
5. BFTS 搜索树的预算和评分权重如何设置？

3.2 输入输出

```
class IdeaSessionRequest(BaseModel):
    seedQuery: str
    paperType: str
    # algorithm | system | benchmark | theory | survey | application
    domain: str | None = None
    constraints: list[str] = []
    mustCiteList: list[str] = []
    maxPapers: int = 120
    maxCandidates: int = 8
    searchBudget: "BFTSBudget" | None = None
```

```

class QueryPlan(BaseModel):
    sessionId: str
    refinedQuestion: str
    queryFamilies: list["QueryFamily"]
    keyConcepts: list[str]
    relatedAreas: list[str]
    excludedScopes: list[str] = []
    paperType: str
    graph2FocusTypes: list[str]
    pathTemplates: list[str]
    rankingWeightProfile: str
    bftsConfig: "BFTSConfig"

class QueryFamily(BaseModel):
    id: str
    query: str
    intent: str
    # core | method | dataset | metric | adjacent | contradiction | survey
    priority: float = 1.0

```

3.3 输出给后续步骤

```

QueryPlan.queryFamilies  -> Step 2 多源检索
QueryPlan.keyConcepts    -> Graph 1 semantic edge 和 Graph 2 entity focus
QueryPlan.pathTemplates  -> Step 4 path seed 模板
QueryPlan.bftsConfig     -> Step 5 搜索树预算

```

四、Step 2: literatureSearch, Graph 1 初建

4.1 目标

Step 2 的名字仍然叫 literatureSearch，但实际职责升级为：

多源文献检索 + 去重合并 + 论文级图谱 Graph 1 初建

这一步不做最终 novelty 判断，也不生成 idea。它只负责拿到足够宽、足够干净、带有图结构的文献池。

4.2 数据来源

来源	作用
Semantic Scholar	标题、摘要、引用、citationCount、TLDR
OpenAlex	concepts、venue、引用网络、跨学科覆盖
arXiv	预印本和近期方法
CrossRef	DOI、出版信息、reference 补全
LocalCorpus	项目本地语料和用户上传文献

第一版可以按成本分级：

MVP: Semantic Scholar + arXiv + LocalCorpus
v5 full: Semantic Scholar + OpenAlex + arXiv + CrossRef + LocalCorpus

4.3 Graph 1 初版结构

节点 = Paper
边 = citation edge + semantic similarity edge + concept overlap edge + author overlap edge

```
class RawPaper(BaseModel):
    id: str
    title: str
    authors: List[str] = []
    abstract: str = ""
    year: int | None = None
    venue: str = ""
    doi: str | None = None
    arxivId: str | None = None
    openalexId: str | None = None
    semanticScholarId: str | None = None
    citationCount: int = 0
    source: List[str] = []
    url: str = ""
    references: List[str] = []
    citedBy: List[str] = []
    concepts: List[str] = []
    retrievalScore: float = 0.0
    relevanceScore: float = 0.0
```

```
class LiteratureGraph(BaseModel):
    sessionId: str
    nodes: List["PaperNode"]
```

```

edges: list["PaperEdge"]
clusters: list["LiteratureCluster"]
buildReport: "LiteratureGraphBuildReport"

class PaperNode(BaseModel):
    paperId: str
    relevanceScore: float
    citationScore: float
    recencyScore: float
    centralityScore: float = 0.0
    clusterId: str | None = None

class PaperEdge(BaseModel):
    id: str
    sourcePaperId: str
    targetPaperId: str
    edgeType: str
    # cites | cited_by | semantic_similar | concept_overlap | author_overlap
    weight: float
    evidence: str = ""

```

4.4 处理流程

```

QueryPlan.queryFamilies
|
|-- parallel multi-source search
|-- DOI / arXiv / title fuzzy dedupe
|-- merge metadata and abstracts
|-- relevance rerank
|   lexical overlap
|   embedding similarity
|   citation / recency signal
|
|-- build Graph 1 initial edges
|   citation edges
|   semantic similarity edges
|   concept overlap edges
|
|-- cluster papers
|   Leiden / HDBSCAN / connected components fallback
|
v
RawPaper[] + LiteratureGraph v0

```

4.5 Step 2 输出

```
RawLiteratureStorage
LiteratureGraphStorage(v0)
```

Step 2 的质量标准：

1. 论文池覆盖不同 query family。
2. 每篇论文有可追溯 source。
3. Graph 1 至少包含 semantic edge。
4. 若引用数据可用，citation edge 必须保留。

五、Step 3: noveltyCheck, Graph 1 精炼和深读

5.1 目标

Step 3 保留 noveltyCheck 名称，但不再是对最终 idea 做黑盒 novelty 打分。

v5 中它负责：

```
在 Graph 1 上做 novelty-aware paper selection,
并对选中论文做 focused deep reading。
```

也就是说，Step 3 的 novelty 是文献空间层面的 novelty：

1. 哪些 cluster 已经过度拥挤？
2. 哪些论文处于 frontier / bridge / contradiction 位置？
3. 哪些组合在文献中少见但有证据支撑？
4. 哪些论文应该进入 Graph 2？

5.2 Graph 1 精炼任务

```
LiteratureGraph v0
|
|-- compute centrality / cluster density / recency
|-- identify core papers
|-- identify cluster representatives
|-- identify frontier papers
|-- identify bridge papers
|-- identify contradiction papers
|-- identify novelty evidence
|-- select papers for deep reading
```

```
|
v
LiteratureGraph v1 + selectedPaperIds
```

论文选择不使用简单 Top-K，而使用配额：

类型	默认数量
core papers	8-12
cluster representatives	每簇 2-3
frontier papers	5-10
bridge papers	5-8
contradiction papers	3-5
must-cite papers	用户指定
final selected papers	30-60

5.3 Focused Deep Reading

Step 3 只深读 Graph 1 选中的论文。

```
class StructuredPaper(BaseModel):
    id: str
    sessionId: str
    title: str
    abstract: str
    authors: List[str] = []
    year: int | None = None
    venue: str = ""
    citationCount: int = 0
    source: List[str] = []

    structuredAbstract: "StructuredAbstract | None" = None
    claims: List["Claim"] = []
    methods: List["MethodMention"] = []
    findings: List["Finding"] = []
    datasets: List[str] = []
    metrics: List[str] = []
    limitations: List[str] = []
    baselines: List[str] = []
    contradictions: List["ContradictionMention"] = []

    graph1Roles: List[str] = []
    # core | representative | frontier | bridge | contradiction | must_cite
```



```
qualityScore: float = 0.0
extractionConfidence: float = 0.0
```

```
class Claim(BaseModel):
    id: str
    text: str
    claimType: str
    # finding | method | comparison | limitation | assumption
    evidenceText: str = ""
    confidence: float = 0.5
```

5.4 Novelty Evidence 模型

```
class NoveltyEvidence(BaseModel):
    id: str
    evidenceType: str
    # sparse_combination | emerging_method | underexplored_dataset |
    # contradiction | weak_baseline | missing_evaluation
    description: str
    paperIds: list[str]
    clusterIds: list[str] = []
    entityHints: list[str] = []
    confidence: float
```

```
class LiteratureMap(BaseModel):
    sessionId: str
    paperCount: int
    selectedPaperIds: list[str]
    clusters: list["LiteratureCluster"]
    frontiers: list["FrontierSignal"]
    gaps: list["GapEvidence"]
    noveltyEvidence: list[NoveltyEvidence]
    selectionReport: "PaperSelectionReport"
```

5.5 Step 3 输出

```
LiteratureGraphStorage(v1)
LiteratureMapStorage
StructuredLiteratureStorage
```

Step 3 完成后，Graph 2 只能读取 `StructuredPaper[]` 和 `LiteratureMap`，不能直接读取全量 `RawPaper[]`。

六、Step 4: gapAnalysis, Graph 2 构建

6.1 目标

Step 4 保留 `gapAnalysis` 名称，但职责升级为：

基于 Graph 1 精选论文构建 Graph 2，并把 `gap/frontier/novelty evidence` 映射到概念级实体。

这一步不再只输出自然语言 gaps，而是输出：

1. ReasoningKG。
2. Graph1-to-Graph2 links。
3. ReasoningPathSeed。
4. 给 Step 5 的 BFTS root nodes。

6.2 Graph 2 结构

节点 = `method` / `problem` / `dataset` / `metric` / `finding` / `limitation` / `mechanism` / `domain`

边 = `uses` / `improves` / `addresses` / `requires` / `contradicts` / `evaluated_on` / `measured_by` / `extends`

```
class KGEntity(BaseModel):
    id: str
    label: str
    entityType: str
    # method | problem | dataset | metric | finding | limitation | mechanism | domain
    sourcePaperIds: List[str] = []
    supportingClaimIds: List[str] = []
    aliases: List[str] = []
    embedding: List[float] | None = None
    confidence: float = 0.5
```

```
class KGRelation(BaseModel):
    id: str
    subjectId: str
    predicate: str
    objectId: str
    sourcePaperIds: List[str] = []
    supportingClaimIds: List[str] = []
    evidenceText: str = ""
    confidence: float = 0.5
```

```
class ReasoningKG(BaseModel):
    sessionId: str
```

```
sourcePaperIds: List[str]
entities: List[KGEntity]
relations: List[KGRelation]
graph1Links: List["GraphEvidenceLink"]
buildReport: "ReasoningKGBuildReport"
```

6.3 Graph 1 到 Graph 2 的映射

```
class GraphEvidenceLink(BaseModel):
    id: str
    graph1SignalType: str
    # gap | frontier | novelty_evidence | contradiction | cluster
    graph1SignalId: str
    entityIds: List[str]
    relationIds: List[str] = []
    supportingPaperIds: List[str]
    confidence: float
```

映射规则:

```
GapEvidence.supportingPaperIds
-> StructuredPaper.limitations / claims / findings
-> KGEntity(problem | limitation | missing_evaluation)
-> GraphEvidenceLink(signalType=gap)
```

```
FrontierSignal.paperIds
-> StructuredPaper.methods / datasets / mechanisms
-> KGEntity(method | dataset | mechanism)
-> GraphEvidenceLink(signalType=frontier)
```

```
NoveltyEvidence(entityHints)
-> entity normalization
-> sparse entity combinations
-> GraphEvidenceLink(signalType=novelty_evidence)
```

6.4 ReasoningPathSeed

Step 4 产出的不是最终路径，而是给 BFTS 使用的种子。

```
class ReasoningPathSeed(BaseModel):
    id: str
    sessionId: str
    templateType: str
    # algorithm | system | benchmark | theory | survey | generic
    anchorEntityIds: List[str]
```

```

LinkedGapIds: List[str] = []
LinkedFrontierIds: List[str] = []
LinkedNoveltyEvidenceIds: List[str] = []
sourcePaperIds: List[str] = []

skeleton: List["PathSeedStep"]
initialScores: "PathSeedScores"
rationale: str

class PathSeedStep(BaseModel):
    stepType: str
    # observation | gap | method | mechanism | prediction | validation
    entityId: str | None = None
    text: str = ""
    required: bool = True

class PathSeedScores(BaseModel):
    noveltyPrior: float
    feasibilityPrior: float
    evidencePrior: float
    graphAlignmentPrior: float

```

不同 paperType 的默认 path seed 模板:

paperType	Seed skeleton
algorithm	Problem -> Limitation -> Method -> Expected Improvement -> Benchmark
system	Need -> Constraint -> Design Choice -> Capability -> Evaluation
benchmark	Missing Evaluation -> Dataset/Metric -> Benchmark Design -> Baselines -> Insights
theory	Assumption -> Contradiction/Gap -> Formal Claim -> Implication -> Proof/Check
survey	Cluster -> Disagreement -> Taxonomy Gap -> Synthesis -> Research Agenda

6.5 Step 4 输出

```

ReasoningKGStorage
GraphEvidenceLinkStorage
PathSeedStorage

```

Step 4 的质量标准：

1. Graph 2 的 `sourcePaperIds` 必须是 Graph 1 `selectedPaperIds` 的子集。
2. 每个 path seed 至少绑定一个 Graph 1 gap/frontier/novelty evidence。
3. 每个 path seed 至少能追溯到一个 StructuredPaper claim。
4. 不允许生成没有图谱实体支撑的 seed。

七、Step 5: ideaBrainstorm, BFTS 搜索树

7.1 目标

Step 5 仍叫 `ideaBrainstorm`，但不再是单轮 LLM 脑暴。

它改成 AI Scientist v2 风格的 BFTS idea search tree：

```
ReasoningPathSeed[]
-> initial IdeaSearchNode[]
-> Best-First Tree Search
    select best expandable nodes
    expand with operators
    run reflection
    run literature probe
    update graph patches
    rescore
    prune
-> IdeaCandidate drafts
```

其中：

```
BFTS = Best-First Tree Search
```

v5 对 AI Scientist v2 的借鉴点：

1. 用树搜索替代单轮生成。
2. 每个节点是一种可继续改进的研究状态。
3. 搜索由 manager 按 score 和预算选择节点扩展。
4. 反思、debug、检索、重写都作为节点扩展的一部分。
5. 最终保留完整 tree trace，方便解释 idea 如何演化。

这里不照搬 AI Scientist v2 的实验执行和论文写作，只把它的 progressive agentic tree-search 思想接到 FAROS 的 idea 阶段。

7.2 搜索树节点

```
class IdeaSearchNode(BaseModel):
    id: str
    sessionId: str
    parentIds: list[str] = []
    generation: int = 0
    operator: str
    # seed | expand_path | reflect | literature_probe | mutate |
    # combine | specialize_experiment | repair_evidence

    pathSeedId: str | None = None
    reasoningPath: "ReasoningPathDraft"
    ideaDraft: "IdeaDraft"

    graphPatchIds: list[str] = []
    literatureProbeIds: list[str] = []
    reflectionIds: list[str] = []

    scores: "IdeaNodeScores"
    status: str
    # open | expanded | pruned | failed | candidate
    failureReason: str = ""
```

```
class ReasoningPathDraft(BaseModel):
    steps: list["ReasoningPathStep"]
    sourceEntityIds: list[str]
    sourceRelationIds: list[str]
    sourcePaperIds: list[str]
    linkedGraph1SignalIds: list[str]
```

```
class ReasoningPathStep(BaseModel):
    stepType: str
    text: str
    entityIds: list[str] = []
    relationIds: list[str] = []
    evidencePaperIds: list[str] = []
```

```
class IdeaDraft(BaseModel):
    title: str
    problem: str
    hypothesisStatement: str
    keyInsight: str
    proposedMethod: str
    expectedOutcome: str
    validationPlan: str
    datasets: list[str] = []
```

```
baselines: list[str] = []
metrics: list[str] = []
risks: list[str] = []
closestPriorWork: list[str] = []
```

```
class IdeaNodeScores(BaseModel):
    novelty: float = 0.0
    feasibility: float = 0.0
    impact: float = 0.0
    specificity: float = 0.0
    evidenceSupport: float = 0.0
    graphGrounding: float = 0.0
    riskControl: float = 0.0
    alignment: float = 0.0
    uncertainty: float = 0.0
    total: float = 0.0
```

7.3 搜索树模型

```
class IdeaSearchTree(BaseModel):
    sessionId: str
    rootNodeIds: list[str]
    nodes: list[IdeaSearchNode]
    edges: list["IdeaSearchEdge"]
    config: "BFTSConfig"
    searchReport: "IdeaSearchReport"
```

```
class IdeaSearchEdge(BaseModel):
    sourceNodeId: str
    targetNodeId: str
    operator: str
    rationale: str = ""
```

```
class BFTSConfig(BaseModel):
    maxNodes: int = 40
    maxIterations: int = 4
    beamWidth: int = 6
    expansionWidth: int = 3
    maxLiteratureProbes: int = 24
    maxReflectionRounds: int = 3
    minEvidenceSupport: float = 0.45
    minGraphGrounding: float = 0.55
    pruneDuplicateThreshold: float = 0.82
    scoreWeights: dict[str, float] = {}
```

7.4 BFTS 主循环

```
async def _step_idea_brainstorm(session, kg, path_seeds, literature_map):
    tree = initialize_tree_from_path_seeds(path_seeds)

    while tree.has_budget():
        frontier = tree.open_nodes()
        selected = select_best_first(
            frontier,
            key=lambda n: n.scores.total + 0.15 * n.scores.uncertainty,
            limit=config.expansionWidth,
        )

        for node in selected:
            children = []
            children += await expand_from_graph_path(node, kg)
            children += await reflect_and_repair(node)
            children += await literature_probe_and_update(node)
            children += await mutate_weak_dimension(node)
            children += await combine_with_neighbor(node, tree)

            for child in children:
                child.scores = await score_node(child, kg, literature_map)
                child = await prune_or_keep(child, tree)
                tree.add_child(node, child)

            node.status = "expanded"

    return extract_candidate_nodes(tree)
```

7.5 节点选择策略

节点优先级：

```
priority =
    0.35 * novelty
+ 0.20 * feasibility
+ 0.15 * impact
+ 0.10 * specificity
+ 0.10 * evidenceSupport
+ 0.10 * graphGrounding
+ 0.15 * uncertainty_bonus
- 0.20 * riskPenalty
```

解释：

1. `novelty` 保证搜索树探索有价值空间。
2. `feasibility` 防止生成空想 idea。

3. `uncertainty_bonus` 鼓励对证据不足但潜力高的节点做文献探测。
4. `graphGrounding` 防止节点脱离 Graph 2。

7.6 搜索树 Operators

Operator	输入	输出	作用
<code>seed</code>	ReasoningPathSeed	IdeaSearchNode	从 Graph 2 path seed 初始化
<code>expand_path</code>	node + ReasoningKG	更完整 reasoning path	补齐 observation/gap/method/validation
<code>reflect</code>	node + critique prompt	ReflectionReport	找逻辑断点、证据缺口、实验漏洞
<code>literature_probe</code>	node + missing evidence	LiteratureProbeResult	在搜索树内追加检索 closest prior work
<code>repair_evidence</code>	node + probe result	evidence-fixed node	修复 novelty 或证据不足
<code>mutate</code>	node + weak dimension	improved node	针对 novelty/feasibility/specificity 等弱项变异
<code>combine</code>	two nodes	synthetic node	组合两个高分节点的互补机制
<code>specialize_experiment</code>	node	experiment-specific node	补 baseline/dataset/metric/success criteria

7.7 Reflection 循环

```
class ReflectionReport(BaseModel):
    id: str
    nodeId: str
    logicalGaps: List[str] = []
    missingEvidence: List[str] = []
    closestPriorWorkRisks: List[str] = []
    feasibilityRisks: List[str] = []
    experimentWeaknesses: List[str] = []
    recommendedOperator: str
    shouldPrune: bool = False
    rationale: str = ""
```

反思 prompt 重点：

1. 这条 idea 是否真的由 Graph 2 路径推出？
2. 哪一步推理缺少来源论文？
3. 是否只是已有论文的重述？
4. 哪些 baseline / dataset / metric 缺失？
5. 下一步应该 literature_probe、mutate、combine 还是 prune？

7.8 LiteratureProbe 循环

Step 5 内部允许重新检索文献，但它不是全量重建 Graph 1，而是做增量证据补丁。

```
class LiteratureProbeQuery(BaseModel):
    nodeId: str
    query: str
    intent: str
    # closest_prior | missing_baseline | dataset_check |
    # contradiction_check | feasibility_check
    maxPapers: int = 8
```

```
class LiteratureProbeResult(BaseModel):
    id: str
    nodeId: str
    query: LiteratureProbeQuery
    papers: List[RawPaper]
    closestPriorWorkIds: List[str] = []
    contradictionPaperIds: List[str] = []
    baselinePaperIds: List[str] = []
    summary: str
    noveltyRisk: float
    shouldUpdateGraph: bool = True
```

增量图补丁：

```
class GraphPatch(BaseModel):
    id: str
    sessionId: str
    sourceNodeId: str
    patchType: str
    # new_prior_work | new_baseline | contradiction | dataset | metric
    addedPaperIds: List[str] = []
    addedEntityIds: List[str] = []
    addedRelationIds: List[str] = []
    affectedNodeIds: List[str] = []
    summary: str
```

规则：

1. LiteratureProbe 找到的论文先进入 ProbeLiteratureStorage 。
2. 只有与当前 node 强相关的论文才形成 GraphPatch 。
3. GraphPatch 可以补充 Graph 2，但不改写 Step 3 的 selectedPaperIds。
4. 最终 candidate 必须标注哪些证据来自前置 Graph，哪些来自搜索树增量检索。

7.9 Pruning 规则

节点会被剪枝的情况：

1. 与已有节点 idea 相似度超过阈值。
2. Graph grounding 低于最低阈值。
3. LiteratureProbe 发现几乎同构的 prior work。
4. Reflection 判断实验不可执行。
5. 连续两轮 repair 后 evidenceSupport 仍不足。

```
class PruneDecision(BaseModel):
    nodeId: str
    shouldPrune: bool
    reason: str
    duplicateOfNodeId: str | None = None
    salvageableParts: list[str] = []
```

7.10 Step 5 输出

```
IdeaSearchTreeStorage
GraphPatchStorage
ProbeLiteratureStorage
CandidateDraftStorage
```

Step 5 完成后，候选不是单轮 LLM 结果，而是搜索树中若干 status=candidate 的节点。

八、Step 6: rankCandidates

8.1 目标

Step 6 保留 rankCandidates，负责最终排序、critique 和可交付包装。

它不再只对单轮候选做 8 维加权，而是：

1. 从 BFTS tree 中抽取 candidate nodes。

2. 对候选进行统一上下文下的 batch ranking。
3. 输出 critique、证据链、实验计划。
4. 生成 final idea package。

8.2 最终候选模型

```
class IdeaCandidate(BaseModel):
    id: str
    sessionId: str
    title: str

    searchNodeId: str
    pathSeedId: str | None = None
    reasoningPathId: str | None = None

    problem: str
    hypothesisStatement: str
    keyInsight: str
    proposedMethod: str
    expectedOutcome: str

    graphEvidence: "CandidateGraphEvidence"
    closestPriorWork: list["PriorWorkComparison"] = []
    experimentSpecs: list["ExperimentSpec"] = []
    risks: list["RiskItem"] = []

    scores: "CandidateScores"
    critique: "IdeaCritique" | None = None
```

```
class CandidateGraphEvidence(BaseModel):
    graph1SignalIds: list[str] = []
    graph2EntityIds: list[str] = []
    graph2RelationIds: list[str] = []
    structuredPaperIds: list[str] = []
    probePaperIds: list[str] = []
    reasoningTrace: list[str] = []
```

```
class PriorWorkComparison(BaseModel):
    paperId: str
    overlap: str
    difference: str
    noveltyRisk: float
```

```
class ExperimentSpec(BaseModel):
    name: str
    objective: str
```

```

datasets: list[str] = []
baselines: list[str] = []
metrics: list[str] = []
protocol: str = ""
successCriteria: list[str] = []
expectedFailureModes: list[str] = []

```

8.3 评分维度

维度	含义
novelty	与 closest prior work 的差异是否实质性
feasibility	实验资源、数据、技术路线是否可行
impact	成功后对领域的潜在影响
specificity	hypothesis、baseline、metric、protocol 是否具体
evidenceSupport	是否有足够论文证据支撑
graphGrounding	是否能追溯到 Graph 1/Graph 2
riskControl	风险是否被识别并可控制
alignment	是否符合用户 query 和 constraints

```

class CandidateScores(BaseModel):
    novelty: float
    feasibility: float
    impact: float
    specificity: float
    evidenceSupport: float
    graphGrounding: float
    riskControl: float
    alignment: float
    total: float

```

8.4 输出

```

class RankedIdeaOutput(BaseModel):
    sessionId: str
    candidates: list[IdeaCandidate]
    rankingReport: "RankingReport"
    searchTreeSummary: "IdeaSearchReport"

class RankingReport(BaseModel):
    scoreMethod: str

```

```
weightProfile: str
topCandidateId: str | None
diversityNotes: str = ""
rejectedCandidateSummary: list[str] = []
nextStepRecommendation: str = ""
```

九、Storage 设计

Storage	内容	生产者	消费者
QueryPlanStorage	QueryPlan	Step 1	Step 2-6
RawLiteratureStorage	RawPaper[]	Step 2	Step 2, Step 3
LiteratureGraphStorage	LiteratureGraph v0/ v1	Step 2, Step 3	Step 3, Step 4
LiteratureMapStorage	clusters/frontiers/ gaps/novelty evidence	Step 3	Step 4-6
StructuredLiteratureStorage	StructuredPaper[]	Step 3	Step 4-6
ReasoningKGStorage	ReasoningKG	Step 4	Step 5-6
PathSeedStorage	ReasoningPathSeed[]	Step 4	Step 5
IdeaSearchTreeStorage	IdeaSearchTree	Step 5	Step 6/API
ProbeLiteratureStorage	搜索树增量文献	Step 5	Step 5-6
GraphPatchStorage	Graph 2 增量补丁	Step 5	Step 5-6
CandidateStorage	IdeaCandidate[]	Step 6	API
SessionTraceStorage	step trace, timing, cost	全局	API

关键约束：

Step 4 构建 Graph 2 时不能直接读取 RawLiteratureStorage 全量论文。
Step 5 LiteratureProbe 可以增量检索，但必须标记为 probe evidence。
Step 6 的最终候选必须同时展示 graph evidence 和 probe evidence。

十、API 端点

方法	路径	说明
POST	/ideas/sessions	创建 session
POST	/ideas/sessions/{id}/start	启动六步 pipeline
GET	/ideas/sessions/{id}/trace	查看步骤追踪
GET	/ideas/sessions/{id}/query-plan	查看 Step 1 输出
GET	/ideas/sessions/{id}/literature/ raw	查看 Step 2 宽检索论文池
GET	/ideas/sessions/{id}/graph/ literature	查看 Graph 1
GET	/ideas/sessions/{id}/literature- map	查看 Step 3 clusters/frontiers/ gaps
GET	/ideas/sessions/{id}/literature/ structured	查看深读论文
GET	/ideas/sessions/{id}/graph/ reasoning	查看 Graph 2
GET	/ideas/sessions/{id}/path-seeds	查看 Step 4 path seeds
GET	/ideas/sessions/{id}/search-tree	查看 BFTS 搜索树
GET	/ideas/sessions/{id}/graph- patches	查看搜索树增量图补丁
GET	/ideas/sessions/{id}/candidates	查看最终候选
POST	/ideas/sessions/{id}/select	选择 candidate 生成 ResearchPlan

十一、执行编排

```
async def run_pipeline(self, session_id: str) -> None:
    session = self._get_session(session_id)

    # Step 1
    query_plan = await self._step_expand_query(session)

    # Step 2: Graph 1 initial build
    raw_papers, lit_graph_v0 = await self._step_literature_search(
```

```

        session=session,
        query_plan=query_plan,
    )

    # Step 3: Graph 1 refinement + focused reading
    lit_map, lit_graph_v1, structured_papers = await self._step_novelty_check(
        session=session,
        query_plan=query_plan,
        raw_papers=raw_papers,
        literature_graph=lit_graph_v0,
    )

    # Step 4: Graph 2 build + path seeds
    kg, path_seeds = await self._step_gap_analysis(
        session=session,
        query_plan=query_plan,
        literature_map=lit_map,
        structured_papers=structured_papers,
    )

    # Step 5: BFTS idea search tree
    search_tree, candidate_drafts = await self._step_idea_brainstorm(
        session=session,
        query_plan=query_plan,
        literature_map=lit_map,
        reasoning_kg=kg,
        path_seeds=path_seeds,
    )

    # Step 6: final ranking
    ranked_output = await self._step_rank_candidates(
        session=session,
        query_plan=query_plan,
        literature_map=lit_map,
        reasoning_kg=kg,
        search_tree=search_tree,
        candidate_drafts=candidate_drafts,
    )

    await self._finalize_session(session, ranked_output)

```

十二、模块拆分

建议新增或改造以下文件：

文件	变更
backend/app/models/idea.py	新增 Graph 1、Graph 2、PathSeed、SearchTree、Candidate 模型
backend/app/modules/idea/service.py	保留六步入口，重写每步内部职责
backend/app/modules/idea/literature_graph.py	Graph 1 构建、聚类、筛选
backend/app/modules/idea/deep_reading.py	selected papers 结构化抽取
backend/app/modules/idea/reasoning_kg.py	Graph 2 构建、实体归一化、关系抽取
backend/app/modules/idea/graph_linker.py	Graph 1 signal 到 Graph 2 entity 映射
backend/app/modules/idea/path_seed.py	paperType path seed 生成
backend/app/modules/idea/bfts_search.py	BFTS 搜索树主循环
backend/app/modules/idea/search_operators.py	reflect、literature_probe、mutate、combine 等 operator
backend/app/modules/idea/idea_scoring.py	节点评分和最终候选评分
backend/app/modules/idea/storage.py	新增各类 storage
backend/app/modules/idea/ideas_api.py	新增 graph/search-tree/path-seeds 端点
backend/app/services/prompts.py	新增 graph extraction、reflection、probe、mutation、ranking prompts

十三、Prompt 设计

13.1 Graph 1 signal prompt

System:

You analyze a literature graph and identify paper-level research-space signals.

User:

Given clusters, papers, abstracts, citations, and relevance scores, identify:

1. core papers
2. representative papers
3. frontier papers
4. bridge papers
5. contradiction papers
6. novelty evidence

Return JSON only.

13.2 Graph 2 extraction prompt

System:

You extract concept-level scientific knowledge from structured papers.

User:

Extract entities and relations.

Allowed entity types:

method, problem, dataset, metric, finding, limitation, mechanism, domain.

Allowed predicates:

uses, improves, addresses, requires, contradicts, evaluated_on,
measured_by, extends, compares_to.

Every entity/relation must cite source paper ids and supporting claim ids.

Return JSON only.

13.3 Path seed prompt

System:

You create graph-grounded reasoning path seeds for scientific ideation.

User:

Given a ReasoningKG and Graph 1 gap/frontier/novelty links, create path seeds.

Each seed must include:

observation, gap, method/mechanism, prediction, validation.

Do not invent unsupported entities.

Return JSON only.

13.4 BFTS reflection prompt

System:

You are the reflection agent inside a best-first idea search tree.

User:

Review this idea node:

- reasoning path
- graph evidence
- current idea draft
- closest prior work if any

Identify logical gaps, missing evidence, novelty risks, feasibility risks,
and recommend exactly one next operator:

literature_probe, mutate, combine, specialize_experiment, repair_evidence, prune.

Return JSON only.

13.5 Literature probe prompt

System:

You generate targeted literature search queries for idea validation.

User:

Given an idea node and its missing evidence, generate search queries for:

1. closest prior work
2. missing baselines
3. contradiction checks
4. dataset/metric feasibility

Return compact query strings and intent labels.

十四、MVP 实现顺序

MVP 1: 六步 contract 跑通

1. 保留现有六步方法名。
2. Step 2 输出 RawPaper + 简化 LiteratureGraph。
3. Step 3 输出 selectedPaperIds + StructuredPaper。
4. Step 4 输出 ReasoningKG + PathSeed。
5. Step 5 用简化 BFTS，至少支持 seed、reflect、mutate。
6. Step 6 输出 ranked candidates。

验收:

一个 seedQuery 能端到端生成 3-5 个候选 idea，
每个候选绑定 searchNodeId、pathSeedId、graph evidence。

MVP 2: 强化双图谱

1. Graph 1 加 citation / semantic / concept edges。
2. Graph 1 加 frontier / bridge / contradiction 识别。
3. Graph 2 加实体归一化。
4. 加 GraphEvidenceLink。
5. path seed 必须绑定 Graph 1 signal。

MVP 3: 强化 BFTS

1. 完整实现 open node frontier。
2. 支持 literature_probe 和 graph_patch。
3. 支持 combine。
4. 支持 duplicate pruning。
5. 支持 tree visualization API。

MVP 4: 强化评价和交付

1. final batch ranking。
 2. closest prior work comparison。
 3. critique report。
 4. experiment spec 完整化。
 5. select candidate 后交给 ResearchPlan / Experiment agent。
-

十五、验证方案

15.1 Contract 验证

1. Step 2 产出的 RawPaper 都有 source。
2. LiteratureGraph 至少包含 PaperNode 和 semantic edge。
3. Step 3 的 StructuredPaper 只来自 selectedPaperIds。
4. Step 4 的 ReasoningKG.sourcePaperIds 是 selectedPaperIds 子集。
5. PathSeed 至少绑定一个 GraphEvidenceLink。
6. Step 5 的 IdeaSearchNode 至少绑定 pathSeedId 或 parentIds。
7. Step 6 的 IdeaCandidate 必须绑定 searchNodeId 和 graphEvidence。

15.2 质量验证

1. Graph 1 selected papers 比 relevance Top-K 覆盖更多 cluster。
2. Graph 2 entity/relation 有 source paper 和 evidence text。
3. BFTS 搜索后的 best candidate 分数高于 seed node。
4. LiteratureProbe 能发现 closest prior work 或确认缺失。
5. Reflection 能有效触发 repair/prune。
6. 最终候选包含 hypothesis、baseline、dataset、metric、risk。

15.3 端到端证据链

每个最终 candidate 都应该能展开：

```
IdeaCandidate
-> IdeaSearchNode
-> ReasoningPathDraft
-> PathSeed
-> ReasoningKG entity/relation
-> StructuredPaper claim
-> RawPaper
-> LiteratureGraph cluster/frontier/gap
```

如果某部分来自 Step 5 增量检索，则展开为：

```
IdeaCandidate
-> IdeaSearchNode
-> LiteratureProbeResult
-> ProbeLiterature
-> GraphPatch
```

十六、当前版本决策

v5 的边界：

1. 做 BFTS idea search tree。
2. 做搜索树内 reflection + literature probe。
3. 做 graph patch，但不在 Step 5 全量重建 Graph 1。
4. 不在 idea 模块内执行实验代码。
5. 不在 idea 模块内自动写完整论文。

v5 优先稳定四个 contract：

```
LiteratureMap.selectedPaperIds
ReasoningKG.graph1Links
ReasoningPathSeed
IdeaSearchNode.searchNodeId + graphEvidence
```

最终目标：

```
让 FAROS idea 模块从“串行 LLM 流程”升级为
“双图谱证据构建 + 搜索树式 idea 演化”的可追溯系统。
```

十七、参考来源

本方案使用以下思路作为设计参考：

1. AI Scientist v2 的 progressive agentic tree-search 和 BFTS 配置方式。
 - arXiv: <https://arxiv.org/abs/2504.08066>
 - GitHub: <https://github.com/SakanaAI/AI-Scientist-v2>
2. 当前工作区已有 v4 双图谱方案。
 - `FAROS-Idea-Module-Plan-v4.md`
3. 当前工作区 SciDER 的 evolutionary idea search 实现。
 - `SciDER/scider/agents/ideation_agent/idea_search.py`